Pipelined MIPS Processor in VHDL

Design, Instruction Encoding, and Python Assembler

Technical Project Report

Team Members	Supervisor
Ibrahim Wael	DR. Hatem Zakaria
Abdelrahman Shehata	
Abdullah Ahmed	
Youssef Walid	

Implementation	Toolchain	ISA Subset
5-Stage Pipeline · VHDL	Python Assembler · ModelSim	19 Instructions (R/I/J)

Table of Contents

1.	Project Overview & Goals	3
2.	High-Level Architecture	3
3.	Pipeline Stages	4
4.	Key VHDL Modules	5
5.	Supported Instruction Set	6
6.	Instruction Formats	7
7.	Worked Encoding Example	8
8.	Opcode Reference	9
9.	Control Unit & Hazard Handling	10
10.	ALU Operations	11
11.	Python Assembler (asm2hex.py)	12
12.	Example Program: Factorial	13
13.	Simulation Workflow	14
14.	Results & Limitations	15
15.	Conclusion & Future Work	16

1. Project Overview & Goals

This project implements a pipelined 32-bit MIPS subset processor in VHDL, designed as a teaching-grade CPU that demonstrates all fundamental pipelining concepts: instruction fetch, decode, execution, memory access, and write-back. A companion Python assembler (asm2hex.py) automates translation from MIPS assembly source code to VHDL-ready machine code constants.

Goals

- Build a fully functional 5-stage pipelined MIPS processor in VHDL
- Support a practical instruction subset: 19 instructions across R, I, and J types
- Load programs via instruction memory (imem.vhd) as a 64-word ROM
- Bridge assembly → machine code with a custom two-pass Python assembler
- Demonstrate hazard handling: load-use stalls, control hazard flushes, and forwarding
- Validate correctness through ModelSim simulation with a dedicated testbench (tb.vhd)

2. High-Level Architecture

The top-level entity **mips_pipeline** (TopLevel.vhd) structurally connects all pipeline stages and supporting units. Data flows from left to right through five pipeline stages separated by four pipeline registers. The forwarding unit and hazard detection unit operate in parallel, monitoring register addresses to eliminate or minimise data and control hazards.

Pipeline Flow

Figure 1 — Five-stage pipeline data flow

Register Labels between Stages

Stage Boundary	Pipeline Register	Contents Latched
IF → ID	IF/ID	Instruction word, PC+4
ID → EX	ID/EX	Control signals, rs/rt data, immediate, register addresses
EX → MEM	EX/MEM	ALU result, write data, control signals, destination register
MEM → WB	MEM/WB	Memory read data, ALU result, destination register, control

3. Pipeline Stages

IF — Instruction Fetch

- Fetch 32-bit instruction at current PC from instruction ROM (imem.vhd)
- Increment PC ← PC + 4 on every normal cycle
- Store fetched instruction and PC+4 into IF/ID pipeline register
- PC is stalled (held) when a load-use hazard is detected by the hazard unit

ID — Instruction Decode

- Decode opcode (bits [31:26]) and funct (bits [5:0]) fields
- Read two source registers (rs, rt) from the 32-entry register file
- Sign-extend or zero-extend the 16-bit immediate based on instruction type
- Compute branch target: PC+4 + (sign_ext_imm << 2) for beq/bne
- Compute jump target: {PC+4[31:28], imm26, 00} for j/jal

EX — Execute

- ALU performs the required operation (ADD, SUB, AND, OR, XOR, NOR, SLT)
- Forwarding muxes select ALU operands from EX/MEM or MEM/WB if needed
- Branch zero flag evaluated; branch taken decision forwarded to hazard unit
- JR resolves jump target from register rs at this stage
- LUI: bypasses ALU, upper immediate shifted to bits [31:16]

MEM — Memory Access

- lw: reads a 32-bit word from data memory (dmem.vhd) at ALU result address
- sw: writes rt register value to data memory at ALU result address
- Non-memory instructions pass through without memory access
- Address for load/store = ALU result from EX stage (base + sign-ext offset)

WB — Write Back

- MemToReg mux selects result: ALU output (most instructions) or memory read (lw)
- RegWrite signal enables writing the result to the destination register
- JAL writes PC+4 (return address) into register $ra ($31)
- LUI result (shifted immediate) written to destination register

4. Key VHDL Modules

The processor is built from eleven VHDL source files. Each file corresponds to a self-contained structural or behavioural entity. All files are connected in the top-level entity **mips_pipeline** inside TopLevel.vhd.

Module / File	Type	Role
pc_reg.vhd	Sequential	32-bit program counter with synchronous stall support
imem.vhd	ROM	64-word instruction memory; initialised with hex machine code
ifid.vhd	Reg	IF/ID pipeline register; supports flush and stall
idiex.vhd	Reg	ID/EX pipeline register; carries control signals + operands
ieximem.vhd	Reg	EX/MEM pipeline register; carries ALU result + memory controls
imemiwb.vhd	Reg	MEM/WB pipeline register; carries write-back data and controls
reg_file.vhd	Sequential	32 × 32-bit registers; 2 read ports, 1 synchronous write port
alu.vhd	Comb	7-operation ALU: AND, OR, ADD, XOR, NOR, SUB, SLT
alu_control.vhd	Comb	Translates ALUOp (2-bit) + funct (6-bit) → 3-bit ALU select
main_control.vhd	Comb	Decodes opcode/funct → full 13-signal control bus
dmem.vhd	Sequential	Data memory; synchronous write, combinational read
forwarding_unit.vhd	Comb	EX/MEM and MEM/WB forwarding path selection to ALU inputs
Hazard_detection.vhd	Comb	Load-use stall detection; control hazard flush signal generation
TopLevel.vhd	Structural	Instantiates and connects all entities into the pipeline

5. Supported Instruction Set

The processor supports 19 MIPS instructions distributed across three encoding types. This constitutes a practical teaching subset; it is intentionally not full MIPS — multiply, divide, and shift instructions are not implemented in this design.

Type	Instructions	Count	Notes
R-type (opcode = 0)	add, sub, and, or, xor, nor, slt, jr	8	Distinguished by 6-bit funct field; destination = rd
I-type (various)	addi, slti, andi, ori, lui, lw, sw, beq, bne	9	16-bit immediate; destination = rt (except sw/branch)
J-type (op 2/3)	j, jal	2	26-bit word address; jal saves PC+4 → $ra

■ *NOT full MIPS: no multiply, divide, or shift instructions (sll, srl, sra) in this design.*

6. Instruction Formats

All MIPS instructions are exactly 32 bits wide. The encoding type is identified by the 6-bit opcode field in bits [31:26]. For R-type instructions (opcode = 0) the 6-bit funct field in bits [5:0] further specifies the operation.

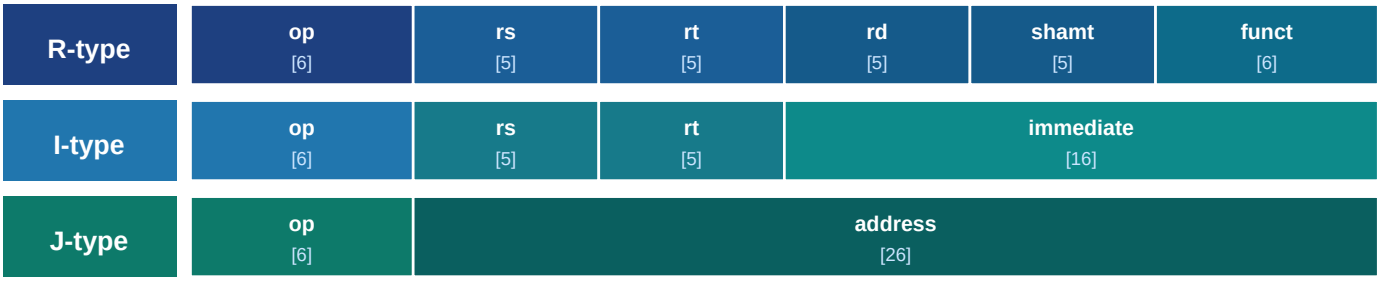

Figure 2 — 32-bit instruction encoding for each format type

Format	Field Layout (bit widths)	Destination	Key Notes
R-type	op[6] rs[5] rt[5] rd[5] shamt[5] funct[6]	rd	shamt=0 for non-shift; funct selects ALU operation
I-type	op[6] rs[5] rt[5] imm[16]	rt	Immediate sign-extended to 32 bits (zero-ext for andi, ori)
J-type	op[6] addr[26]	—	Full PC = {PC+4[31:28], addr[25:0], 00}

7. Worked Encoding Example

The following step-by-step example demonstrates how the assembler encodes a single I-type instruction into its 32-bit machine code representation.

Step	Field	Value	Binary
1 — Identify	Instruction type	addi → I-type	`op(6) | rs(5) | rt(5) | imm(16)`
2 — Opcode	op [31:26]	addi = 8	`001000`
3 — Source	rs [25:21]	$0 = 0	`00000`
4 — Dest	rt [20:16]	$t0 = 8	`01000`
5 — Imm	imm [15:0]	5	`0000000000000101`

Step 6 — Concatenate All Fields

Combine all fields left-to-right (MSB to LSB):

```
op rs rt immediate
001000 | 00000 | 01000 | 0000000000000101

Binary: 0010 0000 0000 1000 0000 0000 0000 0101
Hex: 0x20080005
```

The resulting hex word **0x20080005** is the machine code stored in imem.vhd for the instruction **addi $t0, $0, 5**. This field-extraction and bit-packing logic is the core of asm2hex.py.

8. Opcode Reference

Type	Instruction(s)	Opcode (dec)	Opcode (hex)	Notes
R	`add`	0	0x00	funct = 0x20
R	`sub`	0	0x00	funct = 0x22
R	`and`	0	0x00	funct = 0x24
R	`or`	0	0x00	funct = 0x25
R	`xor`	0	0x00	funct = 0x26
R	`nor`	0	0x00	funct = 0x27
R	`slt`	0	0x00	funct = 0x2A
R	`jr`	0	0x00	funct = 0x08
I	`addi`	8	0x08	Sign-extended immediate
I	`slti`	10	0x0A	Set less-than immediate (signed)
I	`andi`	12	0x0C	Zero-extended immediate
I	`ori`	13	0x0D	Zero-extended immediate
I	`lui`	15	0x0F	Upper 16 bits → rd[31:16]
I	`lw`	35	0x23	Base + sign-ext offset; loads 32-bit word
I	`sw`	43	0x2B	Base + sign-ext offset; stores 32-bit word
I	`beq`	4	0x04	Branch if rs == rt; offset in words
I	`bne`	5	0x05	Branch if rs != rt; offset in words
J	`j`	2	0x02	PC = {PC+4[31:28], addr[25:0], 00}
J	`jal`	3	0x03	Same as j; also writes PC+4 → $ra ($31)

9. Control Unit & Hazard Handling

Main Control Signals

The main control unit (main_control.vhd) decodes the opcode and funct fields from the ID stage and generates the following control signals on every cycle:

Signal	Width	Purpose
RegDst	1-bit	Select destination register: rd (R-type) or rt (I-type)
ALUSrc	1-bit	Select second ALU input: register rt or sign-extended immediate
MemToReg	1-bit	Write-back source: ALU result or data memory output
RegWrite	1-bit	Enable write to register file in WB stage
MemRead	1-bit	Enable data memory read (lw)
MemWrite	1-bit	Enable data memory write (sw)
Branch	1-bit	Enable beq branch condition check
BNE	1-bit	Enable bne branch condition check
Jump	1-bit	Unconditional jump (j / jal)
JAL	1-bit	Jump-and-link — write PC+4 to $ra
JR	1-bit	Jump register — use rs as jump target
LUI	1-bit	Load upper immediate — bypass ALU with shifted imm
ImmZeroExt	1-bit	Zero-extend immediate (andi, ori) instead of sign-extend

Load-Use Hazard

When a load word (lw) instruction is followed immediately by an instruction that uses its destination register, a one-cycle stall is required. The hazard detection unit identifies this condition and:

- Stalls the PC (holds current value for one cycle)
- Freezes the IF/ID pipeline register (preserves fetched instruction)
- Inserts a NOP bubble into the ID/EX register (zeros all control signals)

Control Hazards

When a branch (beq, bne) or jump (j, jal, jr) is taken, the two instructions already fetched into the pipeline must be discarded. The hazard unit flushes:

- IF/ID pipeline register — discards the instruction fetched after the branch
- ID/EX pipeline register — discards the instruction in the decode stage

Forwarding (Data Hazard Resolution)

The forwarding unit (forwarding_unit.vhd) eliminates most data hazards without stalling by routing computed results directly back to ALU inputs:

- EX/MEM forwarding: result from the previous instruction feeds into the current EX stage
- MEM/WB forwarding: result from two instructions ago feeds into the current EX stage

- Priority: EX/MEM forwarding takes precedence over MEM/WB when both match
- Note: forwarding cannot resolve a load-use hazard — a stall is always required

10. ALU Operations

The ALU (alu.vhd) performs seven operations selected by a 3-bit control signal generated by alu_control.vhd. The ALU control unit maps a 2-bit ALUOp signal (from main control) combined with the funct field to the final 3-bit select.

ALU Select	Operation	ALUOp Context	Used By
`000`	AND	R-type funct=0x24	and, andi
`001`	OR	R-type funct=0x25	or, ori
`010`	ADD	R-type funct=0x20	add, addi, lw, sw
`100`	XOR	R-type funct=0x26	xor
`101`	NOR	R-type funct=0x27	nor
`110`	SUB	R-type funct=0x22	sub, beq, bne, slt
`111`	SLT (signed)	R-type funct=0x2A	slt, slti

Special Cases

- Branches (beq, bne): ALU performs SUB on rs and rt. The zero flag output drives the branch condition: beq taken when zero=1, bne taken when zero=0.
- LUI (Load Upper Immediate): bypasses the ALU entirely; the 16-bit immediate is shifted left by 16 bits and zero-padded in the lower 16 bits.
- SLT (Set Less Than): performs signed subtraction and sets result to 1 if rs < rt, otherwise 0.

11. Python Assembler — asm2hex.py

The Python assembler automates translation from MIPS assembly source code to VHDL-ready machine code constants. It implements a classic two-pass assembly algorithm and produces output suitable for direct inclusion in imem.vhd.

Assembly Algorithm

- Pass 1 — Label Collection: scan each non-blank, non-comment line; record label names and their word addresses into a symbol table
- Pass 2 — Encoding: re-scan all instruction lines; look up register names and label addresses; pack fields into 32-bit words using opcode/funct tables
- Output: VHDL mem_array constant assignments, one line per instruction word

Pseudo-operations

Pseudo-op	Expansion	Description
nop	add $0, $0, $0 (0x00000020)	No operation — inserts a pipeline bubble
move rd, rs	add rd, rs, $0	Copy register rs to rd

li rt, imm	addi rt, $0, imm (or lui+ori for large values)	Load immediate value

Usage

```
# Basic usage — outputs hex words to stdout
python asm2hex.py program.asm

# VHDL output mode — generates mem_array constants
python asm2hex.py program.asm --vhdl

# Typical output in imem.vhd:
mem_array(0) <= x"20080005"; -- addi $t0, $0, 5
mem_array(1) <= x"20090001"; -- addi $t1, $0, 1
```

12. Example Program — Factorial (n=5 → 120)

The following program computes 5! = 120 using a simple loop, demonstrating how the forwarding unit and branch mechanism work together.

Assembly Source

```
# Factorial: n=5, result in $t1, stored to dmem[0]
addi $t0, $0, 5 # $t0 = n = 5
addi $t1, $0, 1 # $t1 = result = 1
loop: slt $t2, $0, $t0 # $t2 = (0 < $t0) ? 1 : 0
beq $t2, $0, done # if $t2 == 0, jump to done
add $t1, $t1, $t0 # result *= n (multiply by repeated add)
addi $t0, $t0, -1 # n--
j loop # repeat
done: sw $t1, 0($s2) # store result to data memory
j done # halt (infinite loop)
```

Generated Machine Code

Address	Instruction	Hex Word	Notes
0x00	addi $t0, $0, 5	0x20080005	op=8, rs=0, rt=8, imm=5
0x04	addi $t1, $0, 1	0x20090001	op=8, rs=0, rt=9, imm=1
0x08	slt $t2, $0, $t0	0x0008502A	op=0, rs=0, rt=8, rd=10, funct=0x2A
0x0C	beq $t2, $0, done	0x114B0004	op=4, rs=10, rt=11, offset=4
0x10	add $t1, $t1, $t0	0x01284820	op=0, rs=9, rt=8, rd=9, funct=0x20
0x14	addi $t0, $t0, -1	0x2108FFFF	op=8, rs=8, rt=8, imm=-1 (0xFFFF)
0x18	j loop	0x08000002	op=2, addr=2 (word 2 = 0x08)
0x1C	sw $t1, 0($s2)	0xAE490000	op=0x2B, rs=$s2, rt=9, offset=0

Expected Pipeline Result

- $t1 = 120 after loop completes (5 × 4 × 3 × 2 × 1 through repeated addition)
- dmem[0] = 120 confirmed in ModelSim waveform after sw executes
- Forwarding unit handles RAW hazard in the add → addi sequence without stalling
- PC loops at 'done' address after sw completes (halt pattern)

13. Simulation Workflow

Step	Action	Tool / File	Detail
1	Write Assembly	program.asm	MIPS mnemonics + labels + # comments
2	Run Assembler	asm2hex.py	python asm2hex.py program.asm --vhdl
3	Update Instruction Memory	imem.vhd	Paste generated mem_array constants into ROM
4	Compile Design	ModelSim	Compile all 14 VHDL source files
5	Run Simulation	tb.vhd	Testbench applies clock; run for N cycles
6	Inspect Waveforms	ModelSim waveform	Add registers, PC, memory signals to wave view
7	Verify Results	ModelSim	Check register values, branch targets, dmem

Verification Checklist

Item	What to Verify
ALU Result	Confirm $rd or $rt receives correct arithmetic result after WB stage
Branch Target	Verify PC jumps to correct label on beq/bne with expected condition
Load/Store	Check dmem reads and writes correct 32-bit word at computed address
Forwarding	Ensure no stale register value passed to ALU when forwarding is active
Stall (lw)	Confirm 1-cycle NOP bubble inserted after lw → dependent instruction
JAL	Verify $ra ($31) = PC+4 and PC jumps to the target address
Flush	Confirm 2 instructions discarded from pipeline on taken branch/jump

14. Results & Limitations

Results Achieved

#	Result
1	Fully functional 5-stage pipelined MIPS processor verified in ModelSim
2	EX/MEM and MEM/WB data forwarding eliminates data hazards for all non-load instructions
3	Load-use hazard detection with correct 1-cycle stall and NOP bubble insertion
4	Control hazard flush on all taken branches and jumps (2-cycle penalty)
5	19-instruction subset covering core integer arithmetic, memory, and control flow
6	Two-pass Python assembler with label resolution and VHDL output mode
7	Pseudo-operations: nop, move, li — expanded transparently before encoding
8	ModelSim testbench validates all major instruction and hazard paths

Known Limitations

Limitation	Detail
No cache	Direct memory access only; no L1/L2 cache hierarchy or cache miss handling
No coprocessor	No CP0, no exceptions, no interrupt service routine support
No shift ops	sll, srl, sra not implemented; shamt field is unused in this design
No multiply/divide	mul, div, mfhi, mflo not included in the instruction set
Flush only	Branch mispredictions always incur a 2-cycle flush penalty; no predictor
Fixed IMEM	64-word instruction ROM; no dynamic program loading or memory-mapped I/O

15. Conclusion & Future Work

Summary

This project delivers a complete, teaching-grade MIPS pipelined processor implemented in VHDL, accompanied by a custom Python assembler toolchain. The design successfully demonstrates the five classic RISC pipeline stages and correctly handles data hazards through forwarding, load-use stalls through detection and bubble insertion, and control hazards through pipeline flushing. All 19 supported instructions have been encoded, assembled, loaded, and verified through ModelSim simulation.

Future Work

Direction	Description
Extend the ISA	Add shift instructions (sll, srl, sra) using the shamt field, and implement multiply/divide with HI/LO register
Branch Prediction	Replace the flush-on-branch strategy with a static predictor (always not-taken) or a dynamic 2-bit saturati
CI Integration	Integrate asm2hex.py into a continuous integration pipeline: auto-assemble → simulate with ModelSim in
C Compiler Front-end	Target the assembler from a minimal subset C compiler (or a simple IR-to-assembly back-end) to enable
Cache Subsystem	Add a simple direct-mapped L1 instruction and data cache to demonstrate cache miss stalls and memory
Expanded Register View	Instrument the simulation testbench to produce a full register-file dump at each cycle for automated regre

Ibrahim Wael · Abdelrahman Shehata · Abdullah Ahmed · Youssef Walid

Under Supervision of DR. Hatem Zakaria